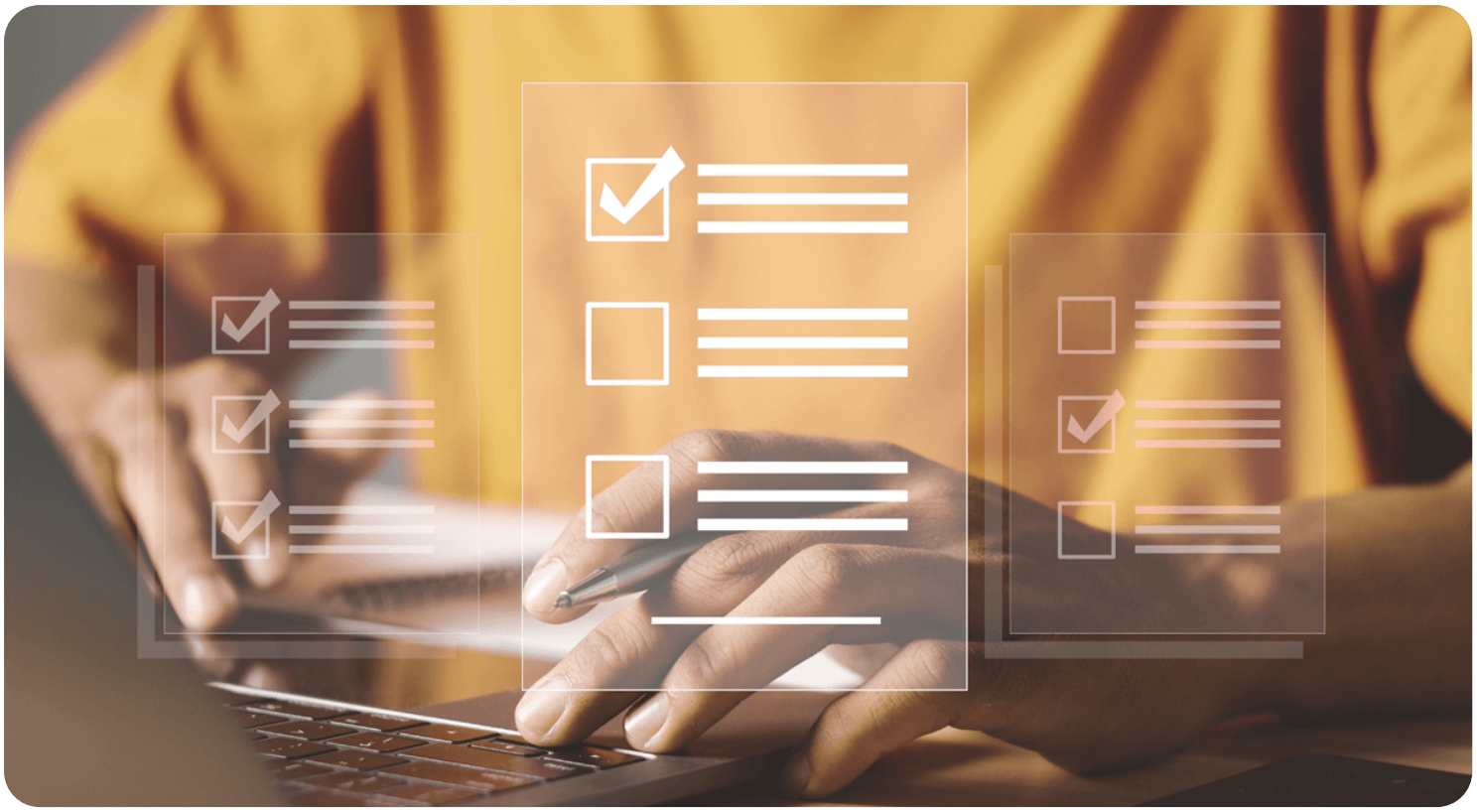
DuocUC | ONLINE



Semana 8

Desarrollo Backend III

(PBY2203)

Nombre estudiante: Raúl Zúñiga	
Asignatura: Desarrollo Backend III	Carrera: Desarrollo de Aplicaciones
Profesor: Gabriel Grobier	Fecha:05-10-2026

Descripción de la actividad

En esta octava semana realizarás una actividad sumativa individual titulada "Desarrollando microservicios y resiliencia en la nube con Spring Cloud". Deberás implementar mecanismos de seguridad con Spring Cloud Security para proteger la infraestructura distribuida, configurar tolerancia a fallos con Resilience4j e integrar un sistema de mensajería asíncrona utilizando Kafka o JMS para habilitar una arquitectura orientada a eventos.

Instrucciones específicas

En función de la actividad formativa de la semana 8 llamada Desarrollando microservicios y resiliencia en la nube con Spring Cloud, deberás avanzar en el proyecto en su continuidad, realizando:

1. Implementar OAuth2.0

Deberás mejorar la seguridad de tu proyecto, implementando protocolo OAuth2.0

2. Dockerizar tus microservicios

Deberás crear una imagen Docker con tus microservicios

3. Configurar docker-compose.yaml en tu proyecto

Deberás orquestar todos los componentes necesarios en una sola configuración para poder lanzar tu aplicación en un entorno Cloud.

Requerimientos del Sistema

Basándote en los microservicios de la semana 6 (datos base: https://github.com/KariVillagran/bank_legacy_data) deberás preparar tus microservicios para un entorno Cloud resiliente y seguro.

Entrega

- **Código Fuente:** Sube el código del proyecto a un repositorio de GitHub con su propuesta técnica
- **Documentación** (README.md): Describe el objetivo del proyecto, la estructura del código y las instrucciones para ejecutar el proyecto.
- **Evidencia de Ejecución:** Proporciona capturas de pantalla o salidas de consola mostrando la ejecución de cada microservicio y los resultados generados.

Desarrollo

1. **Implementar OAuth2.0**

   De acuerdo a lo solicitado para mejorar la seguridad del proyecto se implementó en el paquete bff-web-service en config en la clase SecurityConfig.java la implementación del protocolo OAuth2.0

```java
// Protege los recursos del canal web como OAuth2 Resource Server.
@Configuration
public class SecurityConfig {
    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    UserDetailsService users(PasswordEncoder encoder, @Value("${security.users.web.password}") String web,
            @Value("${security.users.mobile.password}") String mobile,
            @Value("${security.users.atm.password}") String atm) {
        return new InMemoryUserDetailsManager(
                User.withUsername("webuser").password(encoder.encode(web)).roles("WEB").build(),
                User.withUsername("mobileuser").password(encoder.encode(mobile)).roles("MOBILE").build(),
                User.withUsername("atmuser").password(encoder.encode(atm)).roles("ATM").build());
    }

    @Bean
    AuthenticationManager authenticationManager(UserDetailsService users, PasswordEncoder encoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(users);
        provider.setPasswordEncoder(encoder);
        return provider::authenticate;
    }
```

```java
    @Bean
    SecretKey jwtSecretKey(@Value("${security.oauth2.jwt.secret}") String secret) {
        return new SecretKeySpec(Base64.getDecoder().decode(secret), algorithm: "HmacSHA256");
    }

    @Bean
    JwtEncoder jwtEncoder(SecretKey jwtSecretKey) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecretKey));
    }

    @Bean
    JwtDecoder jwtDecoder(SecretKey jwtSecretKey) {
        return NimbusJwtDecoder.withSecretKey(jwtSecretKey).macAlgorithm(MacAlgorithm.HS256).build();
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            String role = jwt.getClaimAsString("role");
            if (role == null || role.isBlank()) {
                return java.util.List.of();
            }
            return java.util.List.of(new org.springframework.security.core.authority.SimpleGrantedAuthority("R
        });
        return converter;
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationConverter jwtAuthenticationConverter,
            @Value("${security.channel-role}") String role) throws Exception {
        return http.csrf(c -> c.disable())
                .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(a -> a.dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
                        .requestMatchers("/auth/login", "/actuator/health").permitAll().anyRequest().hasRole(
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthentic
                .exceptionHandling(e -> e
                        .authenticationEntryPoint(
                                (q, r, x) -> json(r, status: 401, error: "UNAUTHORIZED", message: "Token OAuth
                        .accessDeniedHandler((q, r, x) -> json(r, status: 403, error: "FORBIDDEN",
                                message: "Usuario autenticado sin permisos para este canal")))
                .build();
    }

    private static void json(jakarta.servlet.http.HttpServletResponse r, int status, String error, String mes
            throws java.io.IOException {
        r.setStatus(status);
        r.setContentType("application/json");
        new ObjectMapper().writeValue(r.getOutputStream(),
                Map.of(k1: "status", status, k2: "error", error, k3: "message", message));
    }
}
```

Además se agregó al paquete bff-mobile-service en config en la clase SecurityConfig.java la implementación del protocolo OAuth2.0

```java
// Protege los recursos del canal mobile como OAuth2 Resource Server.
@Configuration
public class SecurityConfig {
    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    UserDetailsService users(PasswordEncoder encoder, @Value("${security.users.web.password}") String web,
            @Value("${security.users.mobile.password}") String mobile,
            @Value("${security.users.atm.password}") String atm) {
        return new InMemoryUserDetailsManager(
            User.withUsername(username: "webuser").password(encoder.encode(web)).roles(...roles: "WEB").bu
            User.withUsername(username: "mobileuser").password(encoder.encode(mobile)).roles(...roles: "MO
            User.withUsername(username: "atmuser").password(encoder.encode(atm)).roles(...roles: "ATM").bu
    }

    @Bean
    AuthenticationManager authenticationManager(UserDetailsService users, PasswordEncoder encoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(users);
        provider.setPasswordEncoder(encoder);
        return provider::authenticate;
    }

    @Bean
    SecretKey jwtSecretKey(@Value("${security.oauth2.jwt.secret}") String secret) {
        return new SecretKeySpec(Base64.getDecoder().decode(secret), algorithm: "HmacSHA256");
    }

    @Bean
    JwtEncoder jwtEncoder(SecretKey jwtSecretKey) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecretKey));
    }

    @Bean
    JwtDecoder jwtDecoder(SecretKey jwtSecretKey) {
        return NimbusJwtDecoder.withSecretKey(jwtSecretKey).macAlgorithm(MacAlgorithm.HS256).build();
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            String role = jwt.getClaimAsString("role");
            if (role == null || role.isBlank()) {
                return java.util.List.of();
            }
            return java.util.List.of(new org.springframework.security.core.authority.SimpleGrantedAuthority("
        });
```

```
        return converter;
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationConverter jwtAuthenticationConverter,
            @Value("${security.channel-role}") String role) throws Exception {
        return http.csrf(c -> c.disable())
                .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(a -> a.dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
                        .requestMatchers("/auth/login", "/actuator/health").permitAll().anyRequest().hasRole(
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthentica
                .exceptionHandling(e -> e
                        .authenticationEntryPoint(
                                (q, r, x) -> json(r, status: 401, error: "UNAUTHORIZED", message: "Token OAuth2
                        .accessDeniedHandler((q, r, x) -> json(r, status: 403, error: "FORBIDDEN",
                                message: "Usuario autenticado sin permisos para este canal")))
                .build();
    }

    private static void json(jakarta.servlet.http.HttpServletResponse r, int status, String error, String mes
            throws java.io.IOException {
        r.setStatus(status);
        r.setContentType(type: "application/json");
        new ObjectMapper().writeValue(r.getOutputStream(),
                Map.of(k1: "status", status, k2: "error", error, k3: "message", message));
    }
}
```

Además se agregó al paquete bff-atm-service en config en la clase SecurityConfig.java la implementación del protocolo OAuth2.0

```java
// Protege los recursos del canal ATM como OAuth2 Resource Server.
@Configuration
public class SecurityConfig {
    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    UserDetailsService users(PasswordEncoder encoder, @Value("${security.users.web.password}") String web,
            @Value("${security.users.mobile.password}") String mobile,
            @Value("${security.users.atm.password}") String atm) {
        return new InMemoryUserDetailsManager(
                User.withUsername(username: "webuser").password(encoder.encode(web)).roles(...roles: "WEB").bu
                User.withUsername(username: "mobileuser").password(encoder.encode(mobile)).roles(...roles: "MC
                User.withUsername(username: "atmuser").password(encoder.encode(atm)).roles(...roles: "ATM").bu
    }

    @Bean
    AuthenticationManager authenticationManager(UserDetailsService users, PasswordEncoder encoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(users);
        provider.setPasswordEncoder(encoder);
        return provider::authenticate;
    }
```

```java
    @Bean
    SecretKey jwtSecretKey(@Value("${security.oauth2.jwt.secret}") String secret) {
        return new SecretKeySpec(Base64.getDecoder().decode(secret), algorithm: "HmacSHA256");
    }

    @Bean
    JwtEncoder jwtEncoder(SecretKey jwtSecretKey) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecretKey));
    }

    @Bean
    JwtDecoder jwtDecoder(SecretKey jwtSecretKey) {
        return NimbusJwtDecoder.withSecretKey(jwtSecretKey).macAlgorithm(MacAlgorithm.HS256).build();
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            String role = jwt.getClaimAsString(claim: "role");
            if (role == null || role.isBlank()) {
                return java.util.List.of();
            }
            return java.util.List.of(new org.springframework.security.core.authority.SimpleGrantedAuthority("
        });
```

```
        return converter;
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationConverter jwtAuthenticationConverter,
            @Value("${security.channel-role}") String role) throws Exception {
        return http.csrf(c -> c.disable())
                .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(a -> a.dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
                        .requestMatchers("/auth/login", "/actuator/health").permitAll().anyRequest().hasRole(
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthentica
                .exceptionHandling(e -> e
                        .authenticationEntryPoint(
                                (q, r, x) -> json(r, status: 401, error: "UNAUTHORIZED", message: "Token OAuth2
                        .accessDeniedHandler((q, r, x) -> json(r, status: 403, error: "FORBIDDEN",
                                message: "Usuario autenticado sin permisos para este canal")))
                .build();
    }

    private static void json(jakarta.servlet.http.HttpServletResponse r, int status, String error, String mess
            throws java.io.IOException {
        r.setStatus(status);
        r.setContentType(type: "application/json");
        new ObjectMapper().writeValue(r.getOutputStream(),
                Map.of(k1: "status", status, k2: "error", error, k3: "message", message));
    }
}

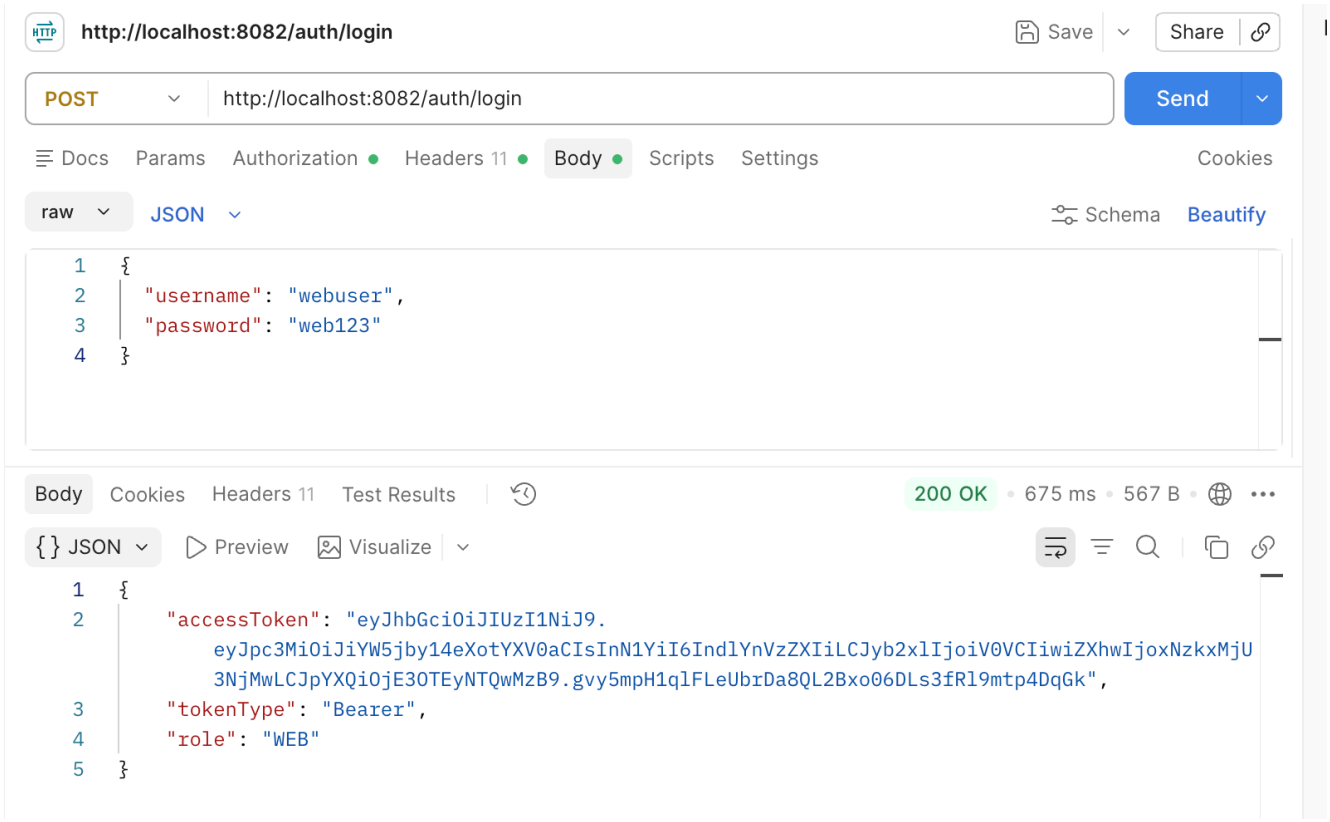
```

Con la aplicación funcionando lo que voy a hacer es abrir POSTMAN y lo que vamos a hacer es obtener el token, para ello debo hacer lo siguiente:

- Crearé una request usando el método POST http://localhost:8082/auth/login
- Ahora me voy a Headers y agregaré en el Content-Type: application/json
- Después me voy a Body y en raw, escribo el siguiente código JSON

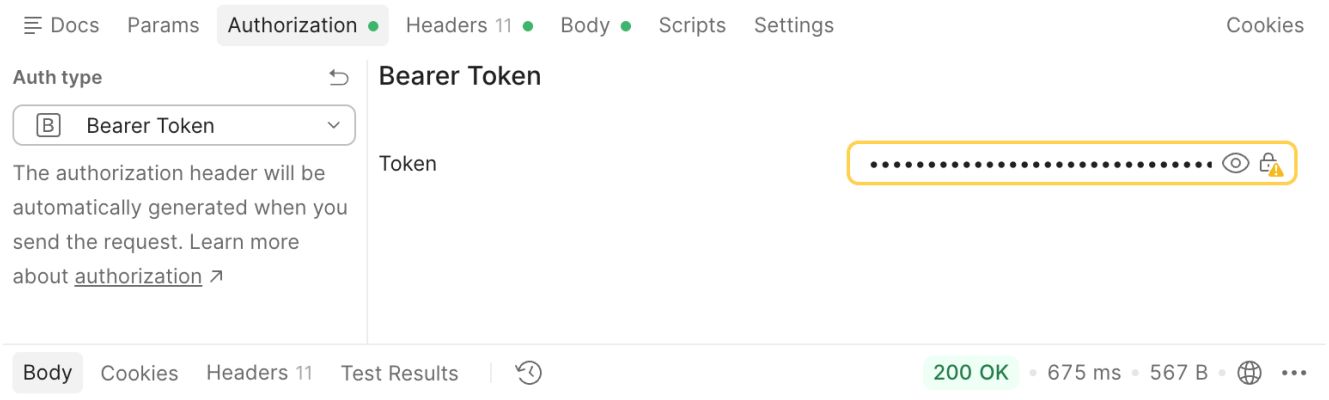
  {
  "username": "webuser",
  "password": "web123"
  }

  Al darle al botón SEND, veré que Postman me devuelve un token de acceso



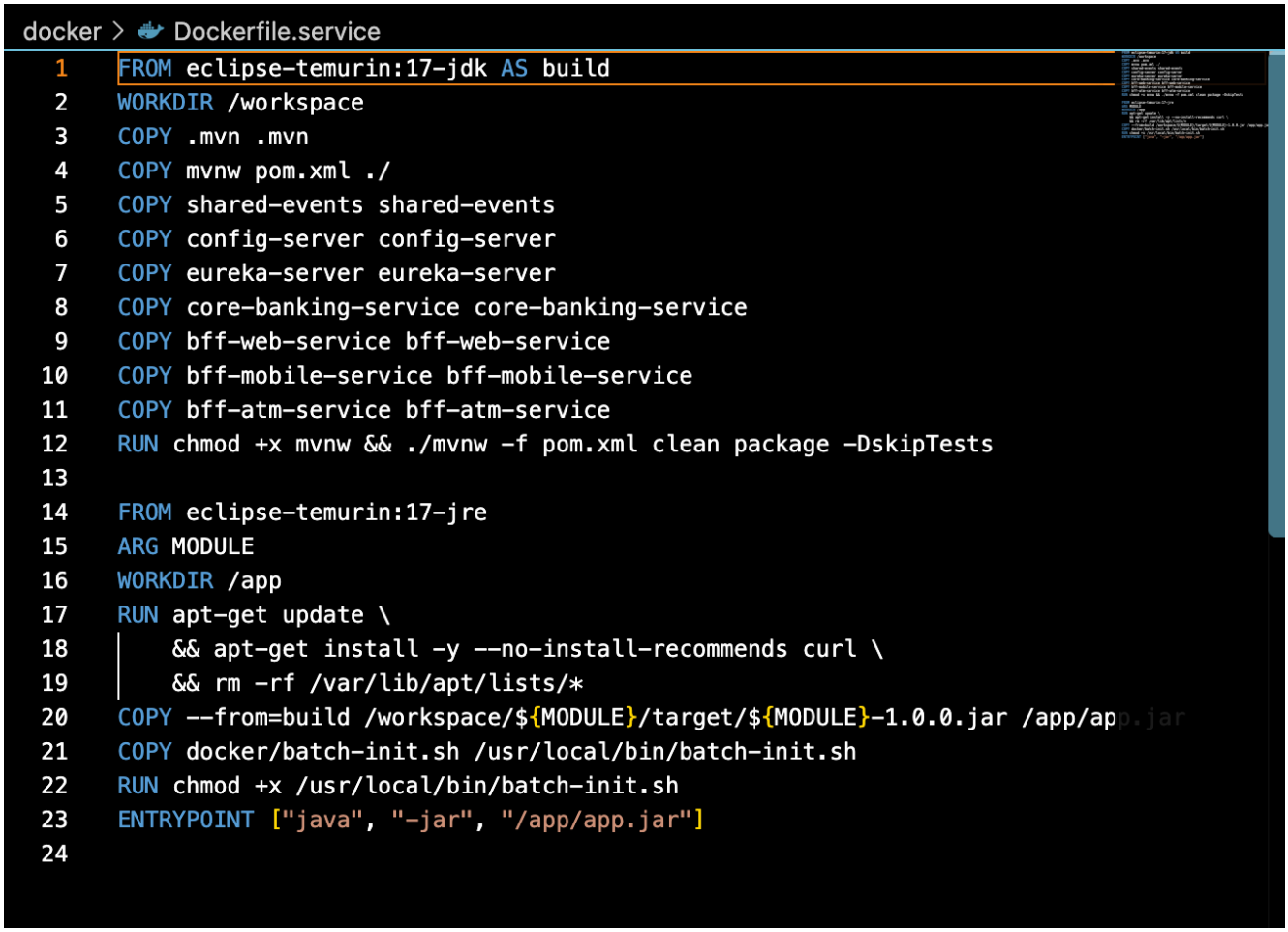
Ahora vamos a usar el token generado para acceder a la información, para ello voy a hacer lo siguiente:

- En Postman debemos ir a donde dice Autorización, seleccionamos donde dice Bearer Token y pegamos el token generado



2. **Dockerizar tus microservicios**

   Se creo dentro del paquete docker una imagen Docker llamada Dockerfile.service este archivo construye la imagen de cada microservicio usando el argumento MODULE.



3. **Configurar docker-compose.yaml en tu proyecto**

   Se creó un archivo docker-compose.yaml el cual permitirá orquestar todos los componentes necesarios en una sola configuración para lanzar el entorno de la aplicación en Cloud. El archivo antes mencionado es el siguiente.

docker-compose.yml

```yaml
services:
  postgres:
    image: postgres:16
    container_name: banco-xyz-postgres
    environment:
      POSTGRES_DB: banco_xyz
      POSTGRES_USER: banco_user
      POSTGRES_PASSWORD: banco_password_local
    ports:
      - "5433:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U banco_user -d banco_xyz"]
      interval: 5s
      timeout: 5s
      retries: 10

  kafka:
    image: apache/kafka:3.9.1
    container_name: banco-xyz-kafka
    ports:
      - "29092:29092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,PLAINTEXT_HOST://:29092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
```

```
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_NUM_PARTITIONS: 3
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list"]
      interval: 10s
      timeout: 10s
      retries: 20

  config-server:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: config-server
    ports:
      - "8888:8888"
    volumes:
      - ./config-repo:/app/config-repo:ro
    depends_on:
      postgres:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8888/actuator/health"]
      interval: 10s
```

```
      timeout: 5s
      retries: 20

  eureka-server:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: eureka-server
    ports:
      - "8761:8761"
    depends_on:
      config-server:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8761/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 30

  batch-init:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: core-banking-service
    entrypoint: ["/usr/local/bin/batch-init.sh"]
    environment:
```

```
      CONFIG_SERVER_URL: http://config-server:8888
      EUREKA_URL: http://eureka-server:8761/eureka/
      KAFKA_BOOTSTRAP_SERVERS: kafka:9092
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/banco_xyz
      SPRING_DATASOURCE_USERNAME: banco_user
      SPRING_DATASOURCE_PASSWORD: banco_password_local
    depends_on:
      postgres:
        condition: service_healthy
      kafka:
        condition: service_healthy
      config-server:
        condition: service_healthy
      eureka-server:
        condition: service_healthy

  core-banking-service:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: core-banking-service
    environment:
      CONFIG_SERVER_URL: http://config-server:8888
      EUREKA_URL: http://eureka-server:8761/eureka/
      EUREKA_INSTANCE_HOSTNAME: core-banking-service
      KAFKA_BOOTSTRAP_SERVERS: kafka:9092
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/banco_xyz
      SPRING_DATASOURCE_USERNAME: banco_user
      SPRING_DATASOURCE_PASSWORD: banco_password_local
      SPRING_BATCH_JOB_ENABLED: "false"
    ports:
      - "8081:8081"
    depends_on:
      kafka:
        condition: service_healthy
      batch-init:
        condition: service_completed_successfully
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8081/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 30

  bff-web-service:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: bff-web-service
    environment:
      CONFIG_SERVER_URL: http://config-server:8888
      EUREKA_URL: http://eureka-server:8761/eureka/
      EUREKA_INSTANCE_HOSTNAME: bff-web-service
      OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl0ZXM=
      OAUTH2_ISSUER: banco-xyz-auth
```

```yaml
    ports:
      - "8082:8082"
    depends_on:
      core-banking-service:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8082/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 30

  bff-mobile-service:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: bff-mobile-service
    environment:
      CONFIG_SERVER_URL: http://config-server:8888
      EUREKA_URL: http://eureka-server:8761/eureka/
      EUREKA_INSTANCE_HOSTNAME: bff-mobile-service
      OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl0ZXM=
      OAUTH2_ISSUER: banco-xyz-auth
    ports:
      - "8083:8083"
    depends_on:
      core-banking-service:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8083/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 30

  bff-atm-service:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: bff-atm-service
    environment:
      CONFIG_SERVER_URL: http://config-server:8888
      EUREKA_URL: http://eureka-server:8761/eureka/
      EUREKA_INSTANCE_HOSTNAME: bff-atm-service
      KAFKA_BOOTSTRAP_SERVERS: kafka:9092
      OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl0ZXM=
      OAUTH2_ISSUER: banco-xyz-auth
    ports:
      - "8084:8084"
    depends_on:
      kafka:
        condition: service_healthy
      core-banking-service:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8084/actuator/health"]
```

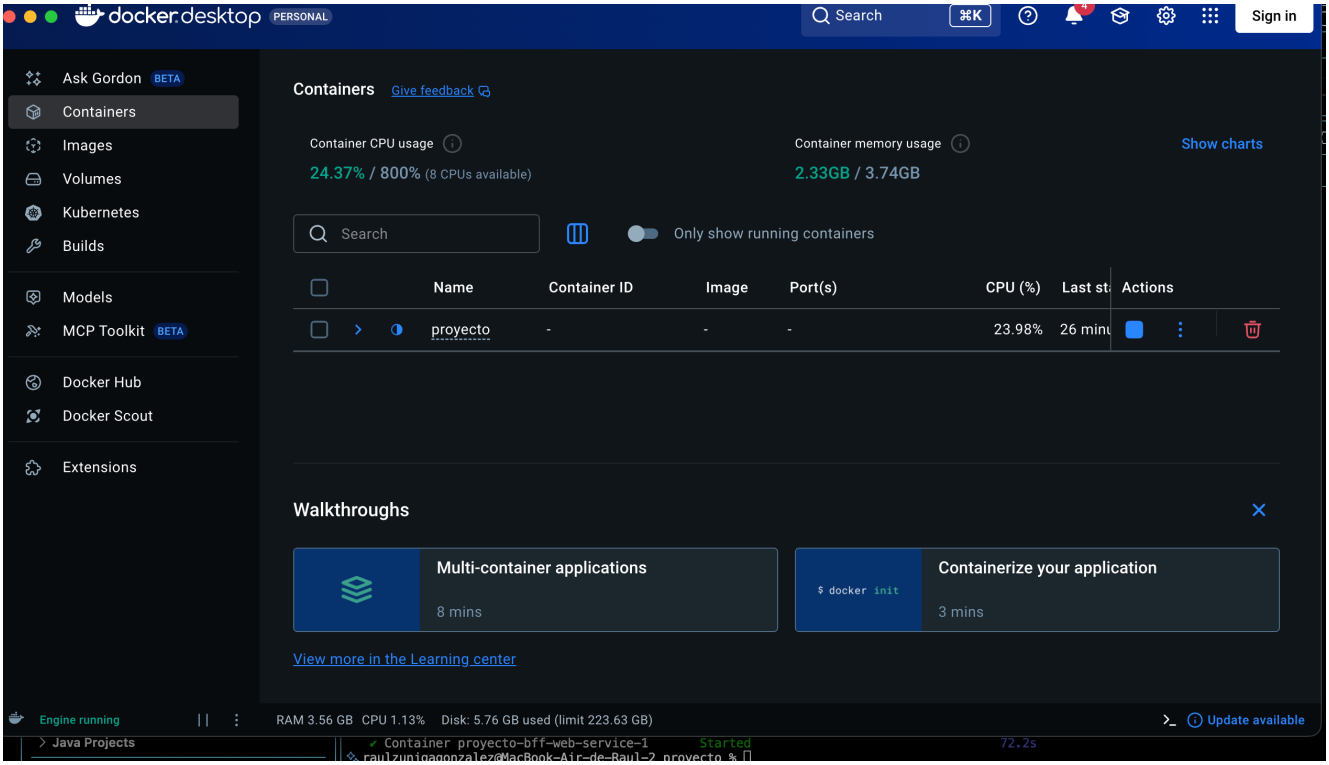
```
200       interval: 10s
201       timeout: 5s
202       retries: 30
203
204   volumes:
205     postgres_data:
206
```

Con el archivo de docker compose creado, lo que debemos hacer en la terminal es escribir el siguiente comando “docker compose up -d –build” y comenzará a ejecutarse docker compose.

```
raulzunigagonzalez@MacBook-Air-de-Raul-2 proyecto % docker compose up -d --build
[+] Running 29/29
 ✔ postgres Pulled                                                          14.5s
 ✔ kafka Pulled                                                             15.7s
[+] Building 123.0s (58/58) FINISHED
 => [internal] load local bake definitions                                   0.0s
 => => reading from stdin 6.94kB                                             0.0s
 => [batch-init internal] load build definition from Dockerfile.service      0.0s
 => => transferring dockerfile: 853B                                         0.0s
 => [bff-web-service internal] load metadata for docker.io/library/eclipse-temurin:17-  1.9s
 => [core-banking-service internal] load metadata for docker.io/library/eclipse-temuri  1.9s
 => [eureka-server internal] load .dockerignore                              0.0s
 => => transferring context: 95B                                             0.0s
 => [bff-web-service build  1/12] FROM docker.io/library/eclipse-temurin:17-jdk@sha256  8.6s
 => => resolve docker.io/library/eclipse-temurin:17-jdk@sha256:5d6042fb8cdc14d614e4e42  0.0s
 => => sha256:97ea7a078a6c8f4aaeb8fa58a0658f6d3631d002252fc264f6be73a8 2.46kB / 2.46kB  0.2s
 => => sha256:70cb2bb6cf6f828366577597e6891d2194b4084dd90082d604d1673f1b6c 158B / 158B  0.2s
 => => sha256:12a4be5f67a4d7dfd447bc299b563bdc9b95b1f453c176a1332e 144.66MB / 144.66MB  6.3s
 => => sha256:787458d0162650daabd0e631a9c7a9b07e87bbe6c06ee05c477799 27.27MB / 27.27MB  2.6s
 => => sha256:06db5ba23986104cd78dcfa853ddf2354c943d2efefc333eca2b44 40.86MB / 40.86MB  6.1s
 => => sha256:5fa77813dc560f299656013d742628128cac51fdfda979349e96a9f09497 392B / 392B  0.4s
 => => extracting sha256:06db5ba23986104cd78dcfa853ddf2354c943d2efefc333eca2b447d4559d  0.6s
 => => extracting sha256:5fa77813dc560f299656013d742628128cac51fdfda979349e96a9f094971  0.0s
 => => extracting sha256:787458d0162650daabd0e631a9c7a9b07e87bbe6c06ee05c4777995a1121f  0.5s
 => => extracting sha256:12a4be5f67a4d7dfd447bc299b563bdc9b95b1f453c176a1332ebb96f75a9  1.1s
 => => extracting sha256:70cb2bb6cf6f828366577597e6891d2194b4084dd90082d604d1673f1b6cb  0.0s
 => => extracting sha256:97ea7a078a6c8f4aaeb8fa58a0658f6d3631d002252fc264f6be73a87eee7  0.0s
 => [bff-atm-service internal] load build context                            3.5s
 => => transferring context: 360.58MB                                        3.4s
 => [batch-init stage-1 1/6] FROM docker.io/library/eclipse-temurin:17-jre@sha256:207  10.4s
 => => resolve docker.io/library/eclipse-temurin:17-jre@sha256:207ecae0b2b104dfc6dfa76  0.0s
 => => sha256:e4a43cb3a392993c9e2c3adb64bf615daa6b46e847d9cca84a9abd62 2.46kB / 2.46kB  0.2s
 => => sha256:0fc159d94561054acfa203f20f782efd944e6757968392e5e0a67ae6cfda 158B / 158B  0.2s
 => => sha256:b4ff5a44a909cc6d314b2fc8a3d748411e5acea8a6b5eb544c4fbd 47.00MB / 47.00MB  7.5s
 => => sha256:1e5320babd49d050bda58cba3fedbe9f9118f2eab0b9c5af2b548a 23.27MB / 23.27MB  2.5s
 => => extracting sha256:1e5320babd49d050bda58cba3fedbe9f9118f2eab0b9c5af2b548af2b7799  0.5s
 => => extracting sha256:b4ff5a44a909cc6d314b2fc8a3d748411e5acea8a6b5eb544c4fbd885f361  1.3s
 => => extracting sha256:0fc159d94561054acfa203f20f782efd944e6757968392e5e0a67ae6cfda7  0.0s
 => => extracting sha256:e4a43cb3a392993c9e2c3adb64bf615daa6b46e847d9cca84a9abd62fc7b2  0.5s
 => [batch-init build  2/12] WORKDIR /workspace                              0.9s
 => [core-banking-service build  3/12] COPY .mvn .mvn                        0.9s
```

```
 => => exporting attestation manifest sha256:b1a627c3cd42eae4ed81e7181cb5088c35d118925  0.0s
 => => exporting manifest list sha256:3b8b0c3115dbda424ab8153fa2dfb20108085c478ceda9da  0.0s
 => => naming to docker.io/library/proyecto-config-server:latest                       0.0s
 => => unpacking to docker.io/library/proyecto-config-server:latest                    0.5s
 => [bff-web-service] exporting to image                                               2.6s
 => => exporting layers                                                                2.0s
 => => exporting manifest sha256:687e6f744e05aa73cab7d4e4c00a69e4674e75c748b08efc1ac5a  0.0s
 => => exporting config sha256:a79b52cac7db0b37d5eeb56a5fb246f65bc4f9f44efa9b1e0ae7fa9  0.0s
 => => exporting attestation manifest sha256:4b0169f82429e9cea31ca688c04db628e44b25b8e  0.0s
 => => exporting manifest list sha256:9ad82e5fc607c980456559b49d0b23acf9ace35a817163c0  0.0s
 => => naming to docker.io/library/proyecto-bff-web-service:latest                     0.0s
 => => unpacking to docker.io/library/proyecto-bff-web-service:latest                  0.6s
 => [config-server] resolving provenance for metadata file                             0.1s
 => [bff-web-service] resolving provenance for metadata file                           0.1s
 => [eureka-server] resolving provenance for metadata file                             0.1s
 => [bff-mobile-service] resolving provenance for metadata file                        0.0s
 => [core-banking-service] resolving provenance for metadata file                      0.0s
 => [bff-atm-service] resolving provenance for metadata file                           0.0s
 => [batch-init] resolving provenance for metadata file                                0.0s
[+] Running 18/18
 ✔ proyecto-config-server                    Built                                    0.0s
 ✔ proyecto-eureka-server                    Built                                    0.0s
 ✔ proyecto-batch-init                       Built                                    0.0s
 ✔ proyecto-core-banking-service             Built                                    0.0s
 ✔ proyecto-bff-web-service                  Built                                    0.0s
 ✔ proyecto-bff-mobile-service               Built                                    0.0s
 ✔ proyecto-bff-atm-service                  Built                                    0.0s
 ✔ Network proyecto_default                  Created                                  0.0s
 ✔ Volume proyecto_postgres_data             Create...                                0.0s
 ✔ Container banco-xyz-kafka                 Healthy                                 62.4s
 ✔ Container banco-xyz-postgres              Healthy                                 25.3s
 ✔ Container proyecto-config-server-1        H...                                    25.0s
 ✔ Container proyecto-eureka-server-1        H...                                    35.5s
 ✔ Container proyecto-batch-init-1           Exit...                                 61.4s
 ✔ Container proyecto-core-banking-service-1 Healthy                                 72.0s
 ✔ Container proyecto-bff-atm-service-1      Started                                 72.3s
 ✔ Container proyecto-bff-mobile-service-1   Started                                 72.3s
 ✔ Container proyecto-bff-web-service-1      Started                                 72.2s
raulzunigagonzalez@MacBook-Air-de-Raul-2 proyecto %
```

Si me voy a docker desktop podré ver que se ha creado el contenedor.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.